

Expected Paragraphs

What did you do?

In this phase of the project, I focused on turning RetailSync from an integrated workflow demo into a more complete and reviewable product experience. Earlier work was about building the foundation, then connecting statement upload, cloud storage, background processing, artifact generation, and QuickBooks operations. In this report period, I worked more on making those workflows clearer, more reliable, and closer to something that could be used for a final project demo.

The biggest progress was in the accounting and month-close workflow. I improved the statement review workspace so a user can upload a bank statement, track processing progress, review extracted entries, inspect artifacts, review suggestions, check validation results, and complete month-close gates inside the app. I also added better statement-list organization with month filtering and status visibility so the user can understand what statements are processing, ready, failed, or completed.

I also worked on the backend statement-processing pipeline, especially around PDF extraction, layout-aware parsing, check detection, validation reports, and saved evidence. In addition to product features, I spent time improving QuickBooks-related workflows, test coverage, documentation, and production-readiness concerns. Overall, this phase was about making RetailSync easier to demo, easier to debug, and easier to understand as a complete system.

How did you do it?

I continued building the project with the same full-stack TypeScript setup. The frontend uses React, Redux Toolkit, and Material UI, while the backend uses Express, MongoDB, Mongoose, and Zod. The project is still organized as a monorepo so shared accounting schemas and status types can be reused between the frontend and backend.

My workflow was usually to start from the real user flow and then connect the frontend, backend, and background logic around it. For the statement workflow, that meant thinking through the full lifecycle: upload the PDF, store it in Google Cloud Storage, create the MongoDB statement record, run background extraction, parse transactions, detect checks, save artifacts, validate totals, show progress in the UI, and allow the user to review the results.

I used Google Cloud Storage for statement files and generated artifacts, MongoDB for workflow records and review state, background task logic for processing, and QuickBooks APIs for accounting integration. I also used Vitest, fixture-based extraction scripts, shared schemas, and documentation updates to keep the project manageable as it became more complex.

What problems did you encounter?

One of the biggest challenges was making the accounting workflow trustworthy. It is not enough for the backend to process a statement; the user also needs to see what happened, what stage the system is in, what artifacts were created, and what still needs review. Because of that, I had to improve the UI around progress tracking, live updates, validation, retry behavior, and review actions.

I also encountered problems with PDF parsing. Bank statements are visually structured documents, but the extracted text is not always organized the same way a human sees it. Some rows can be parsed with

normal text extraction, while other parts need layout-aware parsing to understand columns, check numbers, dates, and amounts. This made validation important because the system needs to compare parsed results against expected totals and balances.

Another challenge was managing external service behavior. Google Cloud Storage, QuickBooks, and AI-assisted proposal generation can all fail or become unavailable depending on configuration, credentials, or rate limits. Because of that, I had to think more carefully about fallback behavior, retries, saved artifacts, and how the UI should explain partial failures instead of making the workflow feel broken.

What Did You Do

What features did you finish?

I finished or significantly improved the statement review and month-close workflow, including statement month filtering, processing status visibility, extracted entry review, suggestion review, validation gates, artifact viewing, check progress, and completion behavior. I also improved the backend processing pipeline with PDF text extraction, layout-aware parsing, check-table extraction, validation reports, check artifacts, and safer retry behavior.

I also improved the QuickBooks side of the project by continuing to polish money operations, live reads, reports, tax-related actions, and standalone QuickBooks navigation. In addition, I updated accounting documentation, workflow notes, wireframes, and tests so the project is easier to understand and evaluate.

Can the features be demoed?

Yes, the features can be demoed. A demo can show login, company-scoped access, opening the statements workspace, filtering statements by month, uploading a statement PDF, watching processing progress, reviewing extracted entries and check artifacts, checking validation results, completing month-close gates, and then showing how the workflow connects to QuickBooks-related accounting operations.

How many of the features are integrated in a way where they can be used together?

Most of the important active features are now integrated in a way where they can be used together. Authentication, company scoping, permissions, statement upload, cloud storage, MongoDB persistence, background processing, artifact generation, validation, review UI, check processing, and QuickBooks-related operations now connect into one larger workflow. Some future areas are still outside the active product scope, but the main demo path works as a connected system instead of separate pieces.

How Did You Do It

Did you use any project templates or tutorials?

I did not use a full project template, but I followed common patterns from React, Express, MongoDB, TypeScript, and cloud-based file upload workflows. I also used official documentation and examples for areas like signed uploads, background processing, QuickBooks API behavior, frontend state management, and testing.

Did you seek help from the instructor or other colleagues?

I mainly implemented the features independently, but I used outside feedback and discussion when I needed to think through workflow design, debugging, architecture decisions, and final project scope. That feedback helped me focus less on adding new modules and more on making the existing accounting and QuickBooks workflows stable and understandable.

Did you use any AI/ML/LLMs?

Yes, I used AI tools during development. I used them for debugging support, code analysis, implementation planning, documentation help, and understanding integration problems faster. I also experimented with AI-assisted proposal generation for accounting suggestions, but I kept deterministic parsing, validation reports, saved artifacts, and fallback behavior so the workflow does not depend only on AI output.

What Problems Did You Encounter

Do you have any concerns about your progress?

My main concern is keeping the final project scope focused. RetailSync now includes authentication, permissions, POS imports, statement upload, PDF extraction, background jobs, validation, review UI, cloud storage, and QuickBooks integration. Because the project is large, I need to prioritize stability, documentation, testing, and a clear demo path instead of adding more major features.

I am also concerned about environment-sensitive behavior. Some parts of the project depend on cloud configuration, OAuth credentials, QuickBooks sandbox access, Google Cloud Storage settings, and local processing tools. I need to make sure the final submission includes enough documentation and fixture-based validation so the project can still be graded clearly even if a third-party service is unavailable.

Do you need any help currently?

The most useful help right now would be feedback on the final demo flow, documentation, and deployment/distribution plan. I would also benefit from feedback on which parts of the project should be emphasized most during grading: the accounting pipeline, QuickBooks integration, architecture, testing, or production-readiness work.